CIS-427 Final Programming Memo

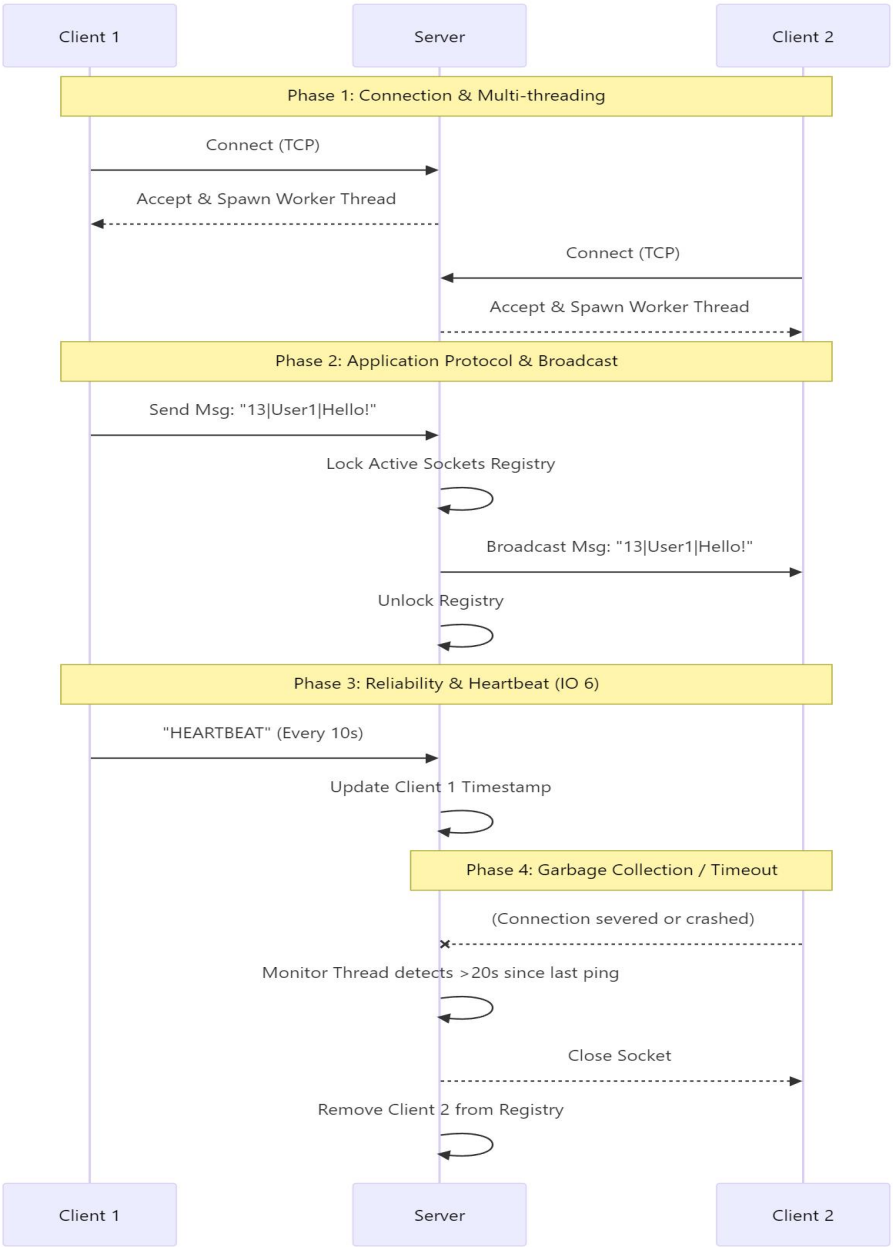
1. Project Contributions

This assignment was completed as a solo project.

2. Design Strategy & Interaction Architecture

The chat application utilizes a centralized, client-server architecture acting as a star topology. Individual clients do not communicate peer-to-peer. Instead, the server acts as the central routing hub. It maintains a stateful registry (a Python dictionary) of all active TCP connections. When a client transmits a message, the server intercepts it, parses it, and broadcasts the byte-stream to all other connected sockets in the registry.

Interaction Diagram:



3. Threading Model

To ensure the server can handle multiple concurrent users and the client can send and receive simultaneously without I/O blocking, the application heavily utilizes multi-threading:

- **Server - Main Listener Thread**: Dedicated strictly to *server.accept()*. It blocks until a new client connects, immediately spawning a worker thread and returning to listen.

- **Server - Client Handler Threads**: One thread is spawned per connected client. It sits in a continuous loop waiting for incoming *.recv()* data to broadcast.

- **Server - Heartbeat Monitor Thread**: A daemon thread running in the background. It wakes up periodically to check the timestamp of every client's last active ping, dropping them if they exceed the 20-second timeout.

- **Client - Main Thread**: Blocks on user *input()* to capture keyboard strokes and transmit them over the socket.

- **Client - Receiver Thread**: A daemon thread that continuously calls *.recv()* to print incoming broadcast messages asynchronously, preventing the terminal from freezing while waiting for a message.

- **Client - Heartbeat Thread**: A daemon thread that silently transmits a *HEARTBEAT* string every 10 seconds.

4. Application Protocol (Message Framing)

To ensure clear data transmission, I implemented a custom application-layer header format: LENGTH|SENDER|MESSAGE.

Framing: Before a client sends a payload, the application measures the string length, prepends the user's name, and separates the metadata using a pipe (|) delimiter.

Parsing: Upon receiving the payload, the client splits the string to isolate the sender and the message. This allows the terminal UI to cleanly format the output as *[Sender]: Message*.

5. Ethics and Data Privacy [CAC 4]

This application utilizes standard TCP byte-streams, meaning data is transmitted in plaintext. Without an encryption layer like TLS/SSL, the network traffic is susceptible

to packet sniffing. To comply with ethical software standards, a hard-coded disclaimer triggers upon launching the client script. It explicitly warns users that the service lacks end-to-end encryption and advises against sharing Personally Identifiable Information (PII) or sensitive financial data.

6. Environment Setup & Code Explanation

The project requires a standard Python 3.x environment using the built-in *socket* and *threading* libraries; no external dependencies are required.

Execution: The server is initialized first via *python server.py*, binding to *localhost* on port 5555. Clients connect by running python *client.py* in separate terminal instances.

State Management: To prevent race conditions during broadcasts or socket cleanup, thread-safe operations are enforced on the server-side using *threading.Lock()* when modifying the active client dictionary.